Atividade: Expressões Regulares com o módulo re

Dados Não Estruturados · Augusto Blois

Exercício 1: Extrair datas em múltiplos formatos

```
import re

texto = "As reuniões serão em 12/03/2024, 15-04-2025 e 22.05.2023."
padrao = r'\b\d{2}[/.-]\d{2}[/.-]\d{4}\b'

datas = re.findall(padrao, texto)
print(datas)
```

Saída: `['12/03/2024', '15-04-2025', '22.05.2023']`

O padrão procura 2 dígitos, um separador, 2 dígitos, outro separador e 4 dígitos. O `[/.-]` aceita barra, ponto ou hífen como separador, e o `\b` garante que a data não está colada em outro número.

Exercício 2: Validar números de telefone brasileiros

```
import re

texto = "Ligue para (83) 98888-7777 ou 11999998888 ou (11)3777-8888."
padrao = r'\(?\d{2}\)?\s?\d{4,5}-?\d{4}'

telefones = re.findall(padrao, texto)
print(telefones)
```

Saída: `['(83) 98888-7777', '11999998888', '(11)3777-8888']`

O `?` deixa os parênteses, o espaço e o hífen opcionais, então o mesmo padrão pega o número formatado e o sem formatação. O `\d{4,5}` aceita tanto celular (5 dígitos) quanto fixo (4 dígitos).

Exercício 3: Extrair URLs de um texto

```
import re

texto = "Acesse https://ufpb.br ou www.google.com ou
http://example.org/teste."
padrao = r'(?:https?://|www\.)\S*[^\s.,;!?]'

urls = re.findall(padrao, texto)
print(urls)
```

Saída: `['https://ufpb.br', 'www.google.com', 'http://example.org/teste']`

A URL começa com `http://`, `https://` ou `www.` e segue até o próximo espaço. O último trecho `[^\s.,;!?]` obriga a URL a terminar em um caractere que não seja pontuação, senão o ponto final da frase entraria junto (ficaria `teste.`).

Exercício 4: Detectar palavras repetidas

```
import re

texto = "Isso é muito muito importante e o o aluno precisa corrigir."
padrao = r'\b(\w+)\s+\1\b'

repeticoes = [m.group() for m in re.finditer(padrao, texto)]
print(repeticoes)
```

Saída: `['muito muito', 'o o']`

O grupo `(\w+)` captura uma palavra e o `\1` exige que a mesma palavra apareça de novo logo depois. Usei `finditer` no lugar de `findall` porque, quando o padrão tem grupo, o `findall` devolve só o grupo (`'muito'`), e o `m.group()` devolve o trecho inteiro (`'muito muito'`).

Exercício 5: Remover hashtags e menções

```
import re

texto = "Amei o evento! #UFPB @usuario #Python"
padrao = r'[#@]\w+\s*'

texto_limpo = re.sub(padrao, '', texto)
print(repr(texto_limpo))
```

Saída: `'Amei o evento! '`

O `[#@]\w+` pega qualquer palavra que começa com # ou @, e o `\s*` remove também o espaço depois dela, para não sobrar espaço acumulado. O `re.sub` troca tudo isso por nada. Usei `repr` só para mostrar o espaço no final, igual à saída esperada.